

```

4      movq    %rsp, %rdi    Save stack pointer
5      pushq   $0xabcd      Push test value
6      popq    %rsp          Pop to stack pointer
7      movq    %rsp, %rax    Set popped value as return value
8      movq    %rdi, %rsp    Restore stack pointer
9      ret

```

我们发现函数总是返回 0xabcd。这表示 popq %rsp 的行为是怎样的？还有什么其他 Y86-64 指令也会有相同的行为吗？

旁注 正确了解细节：x86 模型间的不一致

练习题 4.7 和练习题 4.8 可以帮助我们确定对于压入和弹出栈指针指令的一致惯例。看上去似乎没有理由会执行这样两种操作，那么一个很自然的问题就是“为什么要担心这样一些吹毛求疵的细节呢？”

从下面 Intel 关于 PUSH 指令的文档[51]的节选中，可以学到关于这个一致的重要性的有用的教训：

对于 IA-32 处理器，从 Intel 286 开始，PUSH ESP 指令将 ESP 寄存器的值压入栈中，就好像它存在于这条指令被执行之前。（对于 Intel 64 体系结构、IA-32 体系结构的实地址模式和虚 8086 模式来说也是这样。）对于 Intel® 8086 处理器，PUSH SP 将 SP 寄存器的新值压入栈中（也就是减去 2 之后的值）。（PUSH ESP 指令。Intel 公司。50。）

虽然这个说明的具体细节可能难以理解，但是我们可以看到这条注释说明的是当执行压入栈指针寄存器指令时，不同型号的 x86 处理器会做不同的事情。有些会压入原始的值，而有些会压入减去后的值。（有趣的是，对于弹出栈指针寄存器没有类似的歧义。）这种不一致有两个缺点：

- 它降低了代码的可移植性。取决于处理器模型，程序可能会有不同的行为。虽然这样特殊的指令并不常见，但是即使是潜在的不兼容也可能带来严重的后果。
- 它增加了文档的复杂性。正如在这里我们看到的那样，需要一个特别的说明来澄清这些不同之处。即使没有这样的特殊情况，x86 文档就已经够复杂的了。

因此我们的结论是，从长远来看，提前了解细节，力争保持完全的一致能够节省很多的麻烦。

4.2 逻辑设计和硬件控制语言 HCL

在硬件设计中，用电子电路来计算对位进行运算的函数，以及在各种存储器单元中存储位。大多数现代电路技术都是用信号线上的高电压或低电压来表示不同的位值。在当前的技术中，逻辑 1 是用 1.0 伏特左右的高电压表示的，而逻辑 0 是用 0.0 伏特左右的低电压表示的。要实现一个数字系统需要三个主要的组成部分：计算对位进行操作的函数的组合逻辑、存储位的存储器单元，以及控制存储器单元更新的时钟信号。

本节简要描述这些不同的组成部分。我们还将介绍 HCL (Hardware Control Language, 硬件控制语言)，用这种语言来描述不同处理器设计的控制逻辑。在此我们只是简略地描述 HCL，HCL 完整的参考请见网络旁注 ARCH:HCL。

旁注 现代逻辑设计

曾经，硬件设计者通过描绘示意性的逻辑电路图来进行电路设计（最早是用纸和笔，后来是用计算机图形终端）。现在，大多数设计都是用硬件描述语言 (Hardware Description

Language, HDL)来表达的。HDL 是一种文本表示,看上去和编程语言类似,但是它是用来描述硬件结构而不是程序行为的。最常用的语言是 Verilog,它的语法类似于 C;另一种是 VHDL,它的语法类似于编程语言 Ada。这些语言本来都是用来表示数字电路的模拟模型的。20 世纪 80 年代中期,研究者开发出了逻辑合成(logic synthesis)程序,它可以根据 HDL 的描述生成有效的电路设计。现在有许多商用的合成程序,已经成为产生数字电路的主要技术。从手工设计电路到合成生成的转变就好像从写汇编程序到写高级语言程序,再用编译器来产生机器代码的转变一样。

我们的 HCL 语言只表达硬件设计的控制部分,只有有限的操作集合,也没有模块化。不过,正如我们会看到的那样,控制逻辑是设计微处理器中最难的部分。我们已经开发出了将 HCL 直接翻译成 Verilog 的工具,将这个代码与基本硬件单元的 Verilog 代码结合起来,就能产生 HDL 描述,根据这个 HDL 描述就可以合成实际能够工作的微处理器。通过小心地分离、设计和测试控制逻辑,再加上适当的努力,我们就能创建一个可以工作的微处理器。网络旁注 ARCH:VLOG 描述了如何能产生 Y86-64 处理器的 Verilog 版本。

4.2.1 逻辑门

逻辑门是数字电路的基本计算单元。它们产生的输出,等于它们输入位值的某个布尔函数。图 4-9 是布尔函数 AND、OR 和 NOT 的标准符号,C 语言中运算符(2.1.8 节)的逻辑门下面是对应的 HCL 表达式:AND 用 `&&` 表示,OR 用 `||` 表示,而 NOT 用 `!` 表示。我们用这些符号而不用 C 语言中的位运算符 `&`、`|` 和 `~`,这是因为逻辑门只对单个位的数进行操作,而不是整个字。虽然图中只说明了 AND 和 OR 门的两个输入的版本,但是常见的是它们作为 n 路操作, $n > 2$ 。不过,在 HCL 中我们还是把它们写作二元运算符,所以,三个输入的 AND 门,输入为 a 、 b 和 c ,用 HCL 表示就是 `a && b && c`。

逻辑门总是活动的(active)。一旦一个门的输入变化了,在很短的时间内,输出就会相应地变化。

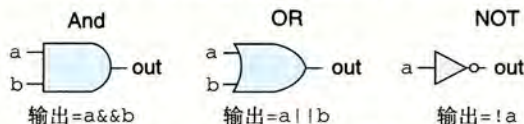


图 4-9 逻辑门类型。每个门产生的输出等于它输入的某个布尔函数

4.2.2 组合电路和 HCL 布尔表达式

将很多的逻辑门组合成一个网,就能构建计算块(computational block),称为组合电路(combinational circuits)。如何构建这些网有几个限制:

- 每个逻辑门的输入必须连接到下述选项之一:1)一个系统输入(称为主输入),2)某个存储器单元的输出,3)某个逻辑门的输出。
- 两个或多个逻辑门的输出不能连接在一起。否则它们可能会使线上的信号矛盾,可能会导致一个不合法的电压或电路故障。
- 这个网必须是无环的。也就是在网中不能有路径经过一系列的门而形成一个回路,这样的回路会导致该网络计算的函数有歧义。

图 4-10 是一个我们觉得非常有用的简单组合电路的例子。它有两个输入 a 和 b ,有唯一的输出 eq ,当 a 和 b 都是 1(从上面的 AND 门可以看出)或都是 0(从下面的 AND 门可以看出)时,输出为 1。用 HCL 来写这个网的函数就是:

```
bool eq = (a && b) || (!a && !b);
```


这段代码简单地定义了位级(数据类型 `bool` 表明了这一点)信号 `eq`, 它是输入 `a` 和 `b` 的函数。从这个例子可以看出 HCL 使用了 C 语言风格的语法, ‘=’ 将一个信号名与一个表达式联系起来。不过同 C 不一样, 我们不把它看成执行了一次计算并将结果放入内存中某个位置。相反, 它只是给表达式一个名字。

 **练习题 4.9** 写出信号 `xor` 的 HCL 表达式, `xor` 就是异或, 输入为 `a` 和 `b`。信号 `xor` 和上面定义的 `eq` 有什么关系?

图 4-11 给出了另一个简单但很有用的组合电路, 称为多路复用器(multiplexor, 通常称为“MUX”)。多路复用器根据输入控制信号的值, 从一组不同的数据信号中选出一个。在这个单个位的多路复用器中, 两个数据信号是输入位 `a` 和 `b`, 控制信号是输入位 `s`。当 `s` 为 1 时, 输出等于 `a`; 而当 `s` 为 0 时, 输出等于 `b`。在这个电路中, 我们可以看出两个 AND 门决定了是否将它们相对应的数据输入传送到 OR 门。当 `s` 为 0 时, 上面的 AND 门将传送信号 `b`(因为这个门的另一个输入是 `!s`), 而当 `s` 为 1 时, 下面的 AND 门将传送信号 `a`。接下来, 我们来写输出信号的 HCL 表达式, 使用的就是组合逻辑中相同的操作:

```
bool out = (s && a) || (!s && b);
```

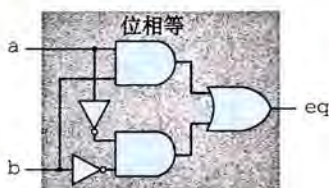


图 4-10 检测位相等的组合电路。当输入都为 0 或都为 1 时, 输出等于 1

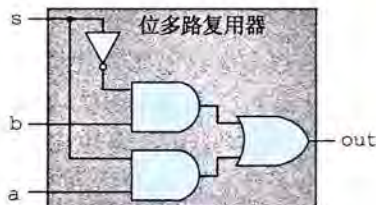


图 4-11 单个位的多路复用器电路。如果控制信号 `s` 为 1, 则输出等于输入 `a`; 当 `s` 为 0 时, 输出等于输入 `b`

HCL 表达式很清楚地表明了组合逻辑电路和 C 语言中逻辑表达式的对应之处。它们都是用布尔操作来对输入进行计算的函数。值得注意的是, 这两种表达计算的方法之间有以下区别:

- 因为组合电路是由一系列的逻辑门组成, 它的属性是输出会持续地响应输入的变化。如果电路的输入变化了, 在一定的延迟之后, 输出也会相应地变化。相比之下, C 表达式只会在程序执行过程中被遇到时才进行求值。
- C 的逻辑表达式允许参数是任意整数, 0 表示 FALSE, 其他任何值都表示 TRUE。而逻辑门只对位值 0 和 1 进行操作。
- C 的逻辑表达式有个属性就是它们可能只被部分求值。如果一个 AND 或 OR 操作的结果只用对第一个参数求值就能确定, 那么就不会对第二个参数求值了。例如下面的 C 表达式:

```
(a && !a) && func(b,c)
```

这里函数 `func` 是不会被调用的, 因为表达式 `(a && !a)` 求值为 0。而组合逻辑没有部分求值这条规则, 逻辑门只是简单地响应输入的变化。

4.2.3 字级的组合电路和 HCL 整数表达式

通过将逻辑门组合成大的网, 可以构造出能计算更加复杂函数的组合电路。通常, 我们设计能对数据字(word)进行操作的电路。有一些位级信号, 代表一个整数或一些控制模

式。例如，我们的处理器设计将包含有很多字，字的大小的范围为4位到64位，代表整数、地址、指令代码和寄存器标识符。

执行字级计算的组合电路根据输入字的各个位，用逻辑门来计算输出字的各个位。例如图4-12中的一个组合电路，它测试两个64位字A和B是否相等。也就是，当且仅当A的每一位都和B的相应位相等时，输出才为1。这个电路是用64个图4-10中所示的单个位相等电路实现的。这些单个位电路的输出用一个AND门连起来，形成了这个电路的输出。

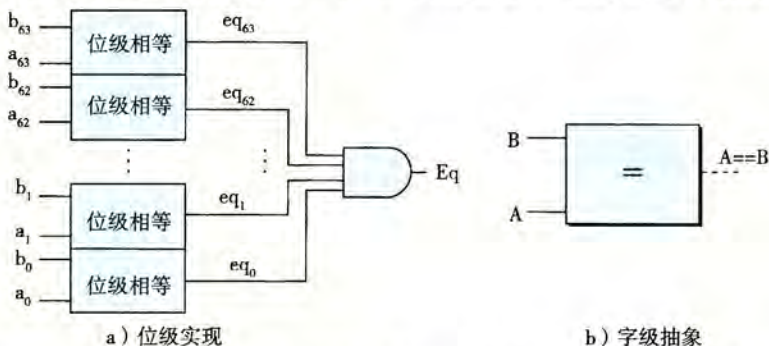


图4-12 字级相等测试电路。当字A的每一位与字B中相应的位均相等时，输出等于1。字级相等是HCL中的一个操作

在HCL中，我们将所有字级的信号都声明为int，不指定字的大小。这样做是为了简单。在全功能的硬件描述语言中，每个字都可以声明为有特定的位数。HCL允许比较字是否相等，因此图4-12所示的电路的函数可以在字级上表达成

```
bool Eq = (A == B);
```

这里参数A和B是int型的。注意我们使用和C语言中一样的语法习惯，‘=’表示赋值，而‘==’是相等运算符。

如图4-12中右边所示，在画字级电路的时候，我们用中等粗度的线来表示携带字的每个位的线路，而用虚线来表示布尔信号结果。

练习题 4.10 假设你用练习题4.9中的异或电路而不是位级的相等电路来实现一个字级的相等电路。设计一个64位字的相等电路需要64个字级的异或电路，另外还要两个逻辑门。

图4-13是字级的多路复用器电路。这个电路根据控制输入位s，产生一个64位的字Out，等于两个输入字A或者B中的一个。这个电路由64个相同的子电路组成，每个子电路的结构都类似于图4-11中的位级多路复用器。不过这个字级的电路并没有简单地复制64次位级多路复用器，它只产生一次!s，然后在每个位的地方都重复使用它，从而减少反相器或非门(inverters)的数量。

处理器中会用到很多种多路复用器，使得我们能根据某些控制条件，从许多源中选出一个字。在HCL中，多路复用函数是用情况表达式(case expression)来描述的。情况表达式的通用格式如下：

```
[
  select1 : expr1;
  select2 : expr2;
  ...
  selectk : exprk;
]
```

这个表达式包含一系列的情况，每种情况 i 都有一个布尔表达式 $select_i$ 和一个整数表达式 $expr_i$ ，前者表明什么时候该选择这种情况，后者指明的是得到的值。

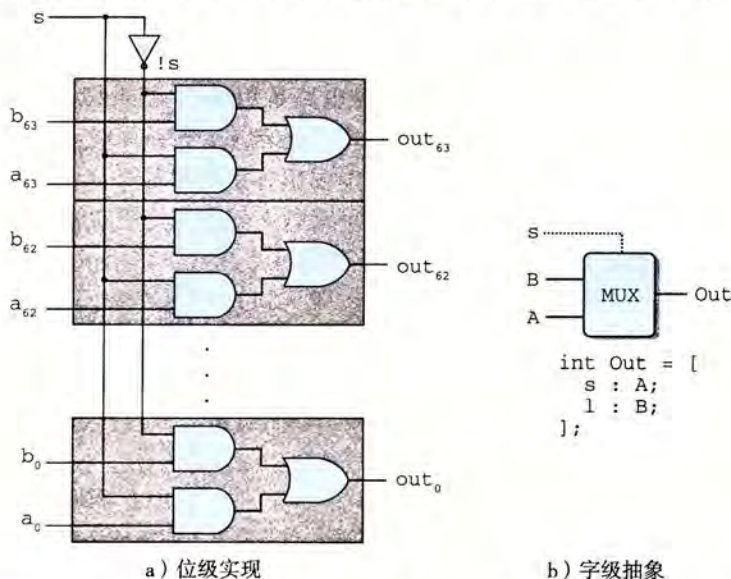


图 4-13 字级多路复用器电路。当控制信号 s 为 1 时，输出会等于输入字 A ，否则等于 B 。HCL 中用情况(case)表达式来描述多路复用器

同 C 的 switch 语句不同，我们不要求不同的选择表达式之间互斥。从逻辑上讲，这些选择表达式是顺序求值的，且第一个求值为 1 的情况会被选中。例如，图 4-13 中的字级多路复用器用 HCL 来描述就是：

```
word Out = [
    s: A;
    1: B;
];
```

在这段代码中，第二个选择表达式就是 1，表明如果前面没有情况被选中，那就选择这种情况。这是 HCL 中一种指定默认情况的方法。几乎所有的情况表达式都是以此结尾的。

允许不互斥的选择表达式使得 HCL 代码的可读性更好。实际的硬件多路复用器的信号必须互斥，它们要控制哪个输入字应该被传送到输出，就像图 4-13 中的信号 s 和 $!s$ 。要将一个 HCL 情况表达式翻译成硬件，逻辑合成程序需要分析选择表达式集合，并解决任何可能的冲突，确保只有第一个满足的情况才会被选中。

选择表达式可以是任意的布尔表达式，可以有任意多的情况。这就使得情况表达式能描述带复杂选择标准的、多种输入信号的块。例如，考虑图 4-14 中所示的四路复用器的图。这个电路根据控制信号 $s1$ 和 $s0$ ，从 4 个输入字 A 、 B 、 C 和 D 中选择一个，将控制信号看作一个两位的二进制数。我们可以用 HCL 来表示这个电路，用布尔表达式描述控制位模式的不同组合：

```
word Out4 = [
    !s1 && !s0 : A; # 00
```

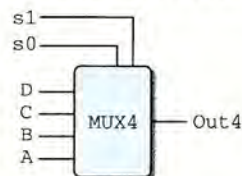


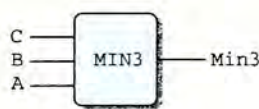
图 4-14 四路复用器。控制信号 $s1$ 和 $s0$ 的不同组合决定了哪个数据输入会被传送到输出


```
!s1      : B; # 01
!s0      : C; # 10
1        : D; # 11
```

];

右边的注释(任何以 # 开头到行尾结束的文字都是注释)表明了 s1 和 s0 的什么组合会导致该情况会被选中。可以看到选择表达式有时可以简化,因为只有第一个匹配的情况才会被选中。例如,第二个表达式可以写成 !s1, 而不用写得更完整 !s1 && s0, 因为另一种可能 s1 等于 0 已经出现在了第一个选择表达式中了。类似地,第三个表达式可以写作 !s0, 而第四个可以简单地写成 1。

来看最后一个例子,假设我们想设计一个逻辑电路来找一组字 A、B 和 C 中的最小值,如下图所示:



用 HCL 来表达就是:

```
word Min3 = [
    A <= B && A <= C : A;
    B <= A && B <= C : B;
    1                  : C;
];
```

 **练习题 4.11** 计算三个字中最小值的 HCL 代码包含了 4 个形如 $X \leq Y$ 的比较表达式。重写代码计算同样的结果,但只使用三个比较。

 **练习题 4.12** 写一个电路的 HCL 代码,对于输入字 A、B 和 C,选择中间值。也就是,输出等于三个输入中居于最小值和最大值之间的那个字。

组合逻辑电路可以设计成在字级数据上执行许多不同类型的操作。具体的设计已经超出了我们讨论的范围。算术/逻辑单元(ALU)是一种很重要的组合电路,图 4-15 是它的一个抽象的图示。这个电路有三个输入:标号为 A 和 B 的两个数据输入,以及一个控制输入。根据控制输入的设置,电路会对数据输入执行不同的算术或逻辑操作。可以看到,这个 ALU 中画的四个操作对应于 Y86-64 指令集支持的四种不同的整数操作,而控制值和这些操作的功能码相对应(图 4-3)。我们还注意到减法的操作数顺序,是输入 B 减去输入 A。之所以这样做,是为了使这个顺序与 subq 指令的参数顺序一致。

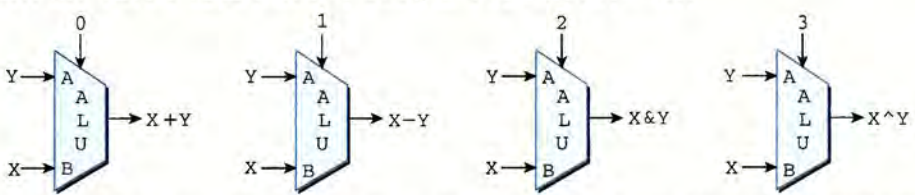
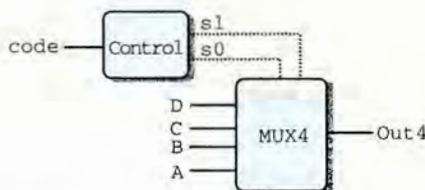


图 4-15 算术/逻辑单元(ALU)。根据函数输入的设置,该电路会执行四种算术和逻辑运算中的一种

4.2.4 集合关系

在处理器设计中,很多时候都需要将一个信号与许多可能匹配的信号做比较,以此来检测正在处理的某个指令代码是否属于某一类指令代码。下面来看一个简单的例子,假设想从一个两位信号 code 中选择高位和低位来为图 4-14 中的四路复用器产生信号 s1 和 s0,

如下图所示：



在这个电路中，两位的信号 `code` 就可以用来控制对 4 个数据字 A、B、C 和 D 做选择。根据可能的 `code` 值，可以用相等测试来表示信号 `s1` 和 `s0` 的产生：

```
bool s1 = code == 2 || code == 3;
bool s0 = code == 1 || code == 3;
```

还有一种更简洁的方式来表示这样的属性：当 `code` 在集合 $\{2, 3\}$ 中时 `s1` 为 1，而 `code` 在集合 $\{1, 3\}$ 中时 `s0` 为 1：

```
bool s1 = code in { 2, 3 };
bool s0 = code in { 1, 3 };
```

判断集合关系的通用格式是：

$$iexpr \text{ in } \{iexpr_1, iexpr_2, \dots, iexpr_k\}$$

这里被测试的值 $iexpr$ 和待匹配的值 $iexpr_1 \sim iexpr_k$ 都是整数表达式。

4.2.5 存储器和时钟

组合电路从本质上讲，不存储任何信息。相反，它们只是简单地响应输入信号，产生等于输入的某个函数的输出。为了产生时序电路(sequential circuit)，也就是有状态并且在这个状态上进行计算的系统，我们必须引入按位存储信息的设备。存储设备都是由同一个时钟控制的，时钟是一个周期性信号，决定什么时候要把新值加载到设备中。考虑两类存储设备：

- 时钟寄存器(简称寄存器)存储单个位或字。时钟信号控制寄存器加载输入值。
- 随机访问存储器(简称内存)存储多个字，用地址来选择该读或该写哪个字。随机访问存储器的例子包括：1)处理器的虚拟内存系统，硬件和操作系统软件结合起来使处理器可以在一个很大的地址空间内访问任意的字；2)寄存器文件，在此，寄存器标识符作为地址。在 IA32 或 Y86-64 处理器中，寄存器文件有 15 个程序寄存器(`%rax~%r14`)。

正如我们看到的那样，在说到硬件和机器级编程时，“寄存器”这个词是两个有细微差别的事情。在硬件中，寄存器直接将它的输入和输出线连接到电路的其他部分。在机器级编程中，寄存器代表的是 CPU 中为数不多的可寻址的字，这里的地址是寄存器 ID。这些字通常都存在寄存器文件中，虽然我们会看到硬件有时可以直接将一个字从一个指令传送到另一个指令，以避免先写寄存器文件再读出来的延迟。需要避免歧义时，我们会分别称呼这两类寄存器为“硬件寄存器”和“程序寄存器”。

图 4-16 更详细地说明了一个硬件寄存器以及它是如何工作的。大多数时候，寄存器都保持在稳定状态(用 x 表示)，产生的输出等于它的当前状态。信号沿着寄存器前面的组合逻辑传播，这时，产生了一个新的寄存器输入(用 y 表示)，但只要时钟是低电位的，寄存器的输出就仍然保持不变。当时钟变成高电位的时候，输入信号就加载到寄存器中，成为下一个状态 y ，直到下一个时钟上升沿，这个状态就一直是寄存器的新输出。关键是寄

寄存器是作为电路不同部分中的组合逻辑之间的屏障。每当每个时钟到达上升沿时，值才会从寄存器的输入传送到输出。我们的 Y86-64 处理器会用时钟寄存器保存程序计数器 (PC)、条件代码 (CC) 和程序状态 (Stat)。

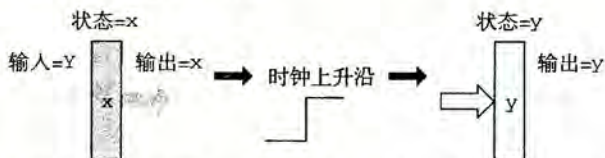
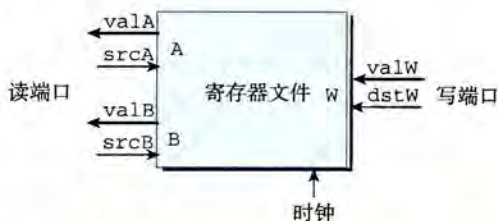


图 4-16 寄存器操作。寄存器输出会一直保持在当前寄存器状态上，直到时钟信号上升。当时钟上升时，寄存器输入上的值会成为新的寄存器状态

下面的图展示了一个典型的寄存器文件：

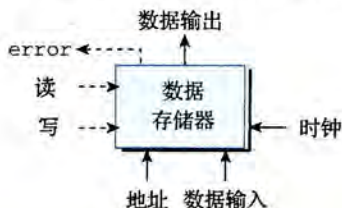


寄存器文件有两个读端口 (A 和 B)，还有一个写端口 (W)。这样一个多端口随机访问存储器允许同时进行多个读和写操作。图中所示的寄存器文件中，电路可以读两个程序寄存器的值，同时更新第三个寄存器的状态。每个端口都有一个地址输入，表明该选择哪个程序寄存器，另外还有一个数据输出或对应应该程序寄存器的输入值。地址是用图 4-4 中编码表示的寄存器标识符。两个读端口有地址输入 `srcA` 和 `srcB` (“source A” 和 “source B” 的缩写) 和数据输出 `valA` 和 `valB` (“value A” 和 “value B” 的缩写)。写端口有地址输入 `dstW` (“destination W” 的缩写)，以及数据输入 `valW` (“value W” 的缩写)。

虽然寄存器文件不是组合电路，因为它有内部存储。不过，在我们的实现中，从寄存器文件读数据就好像它是一个以地址为输入、数据为输出的一个组合逻辑块。当 `srcA` 或 `srcB` 被设成某个寄存器 ID 时，在一段延迟之后，存储在相应程序寄存器的值就会出现在 `valA` 或 `valB` 上。例如，将 `srcA` 设为 3，就会读出程序寄存器 `%rbx` 的值，然后这个值就会出现在输出 `valA` 上。

向寄存器文件写入字是由时钟信号控制的，控制方式类似于将值加载到时钟寄存器。每次时钟上升时，输入 `valW` 上的值会被写入输入 `dstW` 上的寄存器 ID 指示的程序寄存器。当 `dstW` 设为特殊的 ID 值 `0xF` 时，不会写任何程序寄存器。由于寄存器文件既可以读也可以写，一个很自然的问题就是“如果我们试图同时读和写同一个寄存器会发生什么？”答案简单明了：如果更新一个寄存器，同时在读端口上用同一个寄存器 ID，我们会看到一个从旧值到新值的变化。当我们把这个寄存器文件加入到处理器设计中，我们保证会考虑到这个属性的。

处理器有一个随机访问存储器来存储程序数据，如下图所示：



这个内存有一个地址输入，一个写的的数据输入，以及一个读的数据输出。同寄存器文件一样，从内存中读的操作方式类似于组合逻辑：如果我们在输入 `address` 上提供一个地址，并将 `write` 控制信号设置为 0，那么在经过一些延迟之后，存储在那个地址上的值会出现在输出 `data` 上。如果地址超出了范围，`error` 信号会设置为 1，否则就设置为 0。写内存是由时钟控制的：我们将 `address` 设置为期望的地址，将 `data in` 设置为期望的值，而 `write` 设置为 1。然后当我们控制时钟时，只要地址是合法的，就会更新内存中指定的位置。对于读操作来说，如果地址是不合法的，`error` 信号会被设置为 1。这个信号是由组合逻辑产生的，因为所需要的边界检查纯粹就是地址输入的函数，不涉及保存任何状态。

旁注 现实的存储器设计

真实微处理器中的存储器系统比我们在设计中假想的这个简单的存储器要复杂得多。它是由几种形式的硬件存储器组成的，包括几种随机访问存储器和磁盘，以及管理这些设备的各种硬件和软件机制。存储器系统的设计和特点在第 6 章中描述。

不过，我们简单的存储器设计可以用于较小的系统，它提供了更复杂系统的处理器和存储器之间接口的抽象。

我们的处理器还包括另外一个只读存储器，用来读指令。在大多数实际系统中，这两个存储器被合并为一个具有双端口的存储器：一个用来读指令，另一个用来读或者写数据。

4.3 Y86-64 的顺序实现

现在已经有了实现 Y86-64 处理器所需要的部件。首先，我们描述一个称为 SEQ(“sequential”顺序的)的处理器。每个时钟周期上，SEQ 执行处理一条完整指令所需的所有步骤。不过，这需要一个很长的时钟周期时间，因此时钟周期频率会低到不可接受。我们开发 SEQ 的目标就是提供实现最终目的的第一步，我们的最终目的是实现一个高效的、流水线化的处理器。

4.3.1 将处理组织成阶段

通常，处理一条指令包括很多操作。将它们组织成某个特殊的阶段序列，即使指令的动作差异很大，但所有的指令都遵循统一的序列。每一步的具体处理取决于正在执行的指令。创建这样一个框架，我们就能够设计一个充分利用硬件的处理器。下面是关于各个阶段以及各阶段内执行操作的简略描述：

- **取指(fetch)**：取指阶段从内存读取指令字节，地址为程序计数器(PC)的值。从指令中抽取出指令指示符字节的两个四位部分，称为 `icode`(指令代码)和 `ifun`(指令功能)。它可能取出一个寄存器指示符字节，指明一个或两个寄存器操作数指示符 `rA` 和 `rB`。它还可能取出一个四字节常数字 `valC`。它按顺序方式计算当前指令的下一条指令的地址 `valP`。也就是说，`valP` 等于 PC 的值加上已取出指令的长度。
- **译码(decode)**：译码阶段从寄存器文件读入最多两个操作数，得到值 `valA` 和/或 `valB`。通常，它读入指令 `rA` 和 `rB` 字段指明的寄存器，不过有些指令是读寄存器 `%rsp` 的。
- **执行(execute)**：在执行阶段，算术/逻辑单元(ALU)要么执行指令指明的操作(根据 `ifun` 的值)，计算内存引用的有效地址，要么增加或减少栈指针。得到的值我们称为 `valE`。在此，也可能设置条件码。对一条条件传送指令来说，这个阶段会检验条件码和传送条件(由 `ifun` 给出)，如果条件成立，则更新目标寄存器。同样，